

26. Řešení úloh (prohledávání stavového prostoru, rozklad na podúlohy, metody hraní her)

26. Řešení úloh — prohledávání a hraní her

SZZ okruh 26 (FIT BUT). Stavový prostor, prohledávací strategie a herní algoritmy.

Shrnutí

Stavový prostor

- **Stavový prostor (S, O)** — stavy (uzly) a operátory (hrany); **úloha (S₀, G)** = počáteční stav + cílové stavy.
- Kritéria metod: **úplnost** (najde řešení, pokud existuje), **optimálnost** (najde nejlepší), časová/prostorová složitost.

Slepé (neinformované) metody

- **BFS** (fronta, úplná i optimální), **DFS** (zásobník, s modifikací úplná, ne optimální), **DLS**, **IDS** (úplná i optimální), **obousměrné**, **UCS** (respektuje ceny — varianta Dijkstry, úplná i optimální).
- Pro **CSP**: backtracking, forward checking, min-conflicts.

Informované metody

- Ohodnocení $f(s) = g(s)$ (cena dosud) + $h(s)$ (heuristický odhad do cíle).
- **Greedy** ($g=0$; úplná, ne optimální), **A*** (úplná i **optimální**, je-li h **přípustná** — nikdy nepřecení).
- Lokální: **Hill Climbing**, **Simulated Annealing** (umí opustit lokální extrém).

Hraní her

- Rozklad na podúlohy: **AND/OR grafy** (OR = stačí jeden podproblém, AND = všechny).
- **MiniMax** — hráč A maximalizuje (OR), B minimalizuje (AND); **alfa-beta** ořezává zbytečné větve ($\alpha \leq \beta$); **Expectiminimax** pro hry s neurčitostí (očekávané hodnoty).

Prohledávání grafů viz [[okruhy/25-teorie-grafu]]; složitost viz [[okruhy/32-složitost-algoritmu]]; učení viz [[okruhy/27-strojove-uceni]].

[!note] Ke kontrole Zdroj uvádí, že v aktuálních přednáškách se „prohledávání prostoru“ nemusí zkoušet a důraz je na strojovém učení — ověř si rozsah u své komise.

Související syntéza

- [[synthesis/slozitost-napric-algoritmy|Asymptotická složitost napříč algoritmy]] — syntéza
- [[synthesis/zasobnik-napric-obory|Zásobník napříč obory]] — syntéza

Časté otázky u zkoušky

Na co se u tohoto okruhu typicky ptají. Plné odpovědi níže.

Stavový prostor □ [[#Stavový prostor]] - *Formální definice?* □ (S, O) — stavy a operátory; úloha = počáteční + cílové stavy. - *Úplnost vs. optimálnost?* □ Úplná najde řešení, pokud existuje; optimální najde nejlepší (optimální □ úplná).

Neinformované metody □ [[#Slepé (neinformované) metody]] - *BFS vs. DFS?* □ BFS fronta (úplná, optimální, paměťově náročná); DFS zásobník (paměťově úsporná, bez modifikace neúplná). - *UCS?* □ BFS respektující ceny hran (varianta Dijkstry); úplná i optimální.

Informované metody □ [[#Informované metody]] - *A* vs. Greedy?* □ $A^* = g+h$ (úplná i optimální při přípustné h); Greedy jen h ($g=0$, není optimální). - *Heuristická funkce?* □ Odhad ceny do cíle; **přípustná** = nikdy nepřecenění (dolní odhad).

Hraní her □ [[#Hraní her]] - *MiniMax?* □ Hráč na tahu maximalizuje, soupeř minimalizuje; rekurzivně do hloubky. - *Alfa-beta?* □ Ořez větví, když $\alpha \geq \beta$ □ neprohledává se část stromu (stejný výsledek, méně uzlů).

Plné znění (ke studiu)

Stavový prostor

Graf, kde uzly představují jednotlivé stavy úlohy a hrany představují použité operátory. Je to dvojice (S, O) , kde:

- S - množina stavů úlohy - **uzly** (vrcholy) grafu.
- O - množina operátorů - **hrany** grafu.

Úloha

Dvojice (S_0, G) , $S_0 \in S$ a $G \subseteq S$ kde:

- S_0 - **počáteční** stav.
- G - **množina** všech koncových/cílových stavů.

Hodnotící kritéria prohledávacích metod

- **Úplnost** - Algoritmus je úplný, když vždy najde řešení, pokud daná úloha řešení má. Každá úplná metoda pro **CSP** (Constraint Satisfaction Problem - záleží pouze na nalezení cílového stavu, posloupnost operátorů je irelevantní) je **optimální**.
- **Optimálnost** - Pokud existuje více možných řešení, metoda najde to **nejlepší**. Optimální metoda je tedy **vždy úplná**.
- **Paměťová/Prostorová a časová složitost** - Např. lineární - $O(n)$, kvadratická $O(n^2)$, lineární $O(n \cdot \log n)$, exponenciální $O(2^n)$... (Pro více viz Typické příklady časové složitosti.)

Metody řešení úloh prohledáváním stavového prostoru

- **Neinformované/Slepé metody:** Nevyužívají **žádné** informace, které by mohly **usnadnit** nalezení řešení. Používají se pouze pokud o řešení úlože skutečně **žádné** informace **nemáme**.
- **Informované metody** - Využívají nějaké informace (**heuristické** funkce) o řešení úlože, které mohou **usnadňovat** nebo **umožňovat** jejich řešení.

- **Metody lokálního prohledávání** - Místo **systematického** prohledávání stavového prostoru prohledávají pouze **okolí aktuálního stavu**. Vhodné pro řešení optimalizačních problémů. **Nemusí** být **úplné** ani **optimální**.

Pojmy

- **Expanze uzlu** - Určení všech jeho bezprostředních následovníků - uzlů spojených hranou s expandovaným uzlem (postupná aplikace všech operátorů na uzel).
- **Ohodnocení uzlu** - Je dáno součtem **cen přechodů** (součet ohodnocení hran po cestě) z kořenového uzlu do tohoto uzlu.

Slepé metody

Prohledávání do šířky - Breadth First Search (BFS)

Úplná a optimální. Používá **frontu** pro ještě nezpracované uzly - **OPEN** a **seznam** pro již zpracované uzly - **CLOSED**.

1. Sestroj frontu OPEN a seznam CLOSED (pro variantu s CLOSED). Do OPEN vlož počáteční uzel.
2. Je-li fronta OPEN prázdná, pak úloha **nemá řešení**, jinak pokračuj.
3. Vyber z **čela** fronty OPEN uzel.
4. Je-li **cílovým**, ukonči prohledávání jako **úspěšné** a **vrať cestu**, jinak pokračuj.
5. Expanduj uzel (zde lze také testovat na to, jestli je uzel cílový), jeho následovníky co **nejsou** v OPEN ani v CLOSED, umísti do fronty OPEN, expandovaný uzel do CLOSED. Vrať se na bod 2.

Složitost je stejná časově i prostorově a závisí na **faktoru větvení - b** a hloubce **cílového uzlu - d**:

- **bez modifikace** v bodě 5: $O(b^{(d+1)})$,
- **s modifikací** v bodě 5: $O(b^{(d)})$.

Prohledávání do hloubky - Depth First Search (DFS)

1. Sestroj **zásobník** OPEN a umísti do něj počáteční uzel.
2. Je-li OPEN **prázdný**, pak úloha **nemá řešení**, jinak pokračuj.
3. Vyber z vrcholu OPEN první uzel.
4. Je-li uzel uzlem **cílovým**, ukonči prohledávání jako **úspěšné** a vrať cestu, jinak pokračuj.
5. Expanduj vybraný uzel a do OPEN vlož všechny jeho **bezprostřední** následníky (modifikace: bezprostředního následníka vlož na zásobník jen pokud tam **ještě není** a pokud **není předkem** právě expandovaného uzlu). Vrať se na bod 2.

Metoda bez modifikace **není úplná ani optimální** (do nekonečna se můžeme pohybovat v cyklu). Po modifikaci **je** metoda **úplná** ale stále **není optimální**.

- **bez modifikace**: časově i prostorově má nekonečnou časovou složitost,
- **s modifikací**: časová složitost je $O(b^m)$ a prostorová složitost je $O(m)$, kde **b** je faktor větvení a **m** je počet **různých stavů**. U stromové struktury je prostorová složitost $O(b \cdot d)$, kde d je hloubka.

Prohledávání do omezené hloubky - Depth Limited Search (DLS)

Jedná se o metodu DFS, která prohledává pouze do omezené hloubky od počátečního uzlu. **Není úplná ani optimální.**

1. Sestroj **zásobník** OPEN a umísti do něj počáteční uzel.
2. Je-li OPEN **prázdné**, pak úloha **nemá řešení**. Jinak pokračuj.
3. Vyber z vrcholu OPEN první uzel.
4. Je-li vybraný uzel uzlem **cílovým**, ukonči prohledávání jako **úspěšné** a vrať cestu. Jinak pokračuj.
5. Je-li **hloubka** vybraného uzlu **menší** než zadaná **maximální hloubka**, tak tento uzel expanduj a do OPEN vlož všechny následovníky, kteří tam ještě nejsou. Vrať se na bod 2.

Složitost závisí na prohledávané hloubce, **b** - faktor větvení, **l** - prohledávaná hloubka:

- časová: $O(b^l)$,
- prostorová: $O(b \cdot l)$.

Prohledávání do omezené hloubky s postupným zanořováním - Iterative Deepening Search (IDS)

Metoda je **úplná i optimální**.

1. Nastav aktuální maximální hloubku na **1**.
2. Zavolej proceduru **DLS** s omezením na nynější hloubku.
3. Skončí-li **DLS** s úspěchem, ukonči prohledávání s úspěchem a vrať cestu.
4. Skončí-li **DLS** s neúspěchem, tak pokud v DLS **nebyl alespoň jeden uzel expandován z důvodu dosažení maximální hloubky**, inkrementuj maximální hloubku a vrať se na bod 2. Jinak ukonči prohledávání jako **neúspěšné**.

Složitost je dána maximální hloubkou, **b** - faktor větvení, **d** - maximální prohledávaná hloubka

- časová: $O(b^{d+1})$,
- prostorová: $O(b \cdot l)$.

Obousměrné prohledávání - Bidirectional Search (BS)

Metoda je **úplná i optimální**. Metodu lze použít pouze na řešení úloh s **reverzibilními** operátory (např. **Loydova osmička**). Současně prohledává prostor **od počátečního stavu i od cílového stavu** a hledá uzel, ve kterém se obě prohledávání setkají.

[[media/szz-26/media/image5.png]]

Časová i prostorová složitost metody je stejná, **b** - faktor větvení, **d** - hloubka řešení:

- $O(2 \cdot b^{(d/2)})$, což odpovídá $O(b^{(d/2)})$.

Prohledávání do šířky s respektováním cen přechodů - Uniform Cost Search (UCS)

Metoda je **úplná i optimální**. Není vhodná pro úlohy, kde **optimální řešení** leží na **málo cestách s vysokou cenou** (prohledává se zbytečně mnoho cest s malými cenami). Jedná se o variantu Dijkstrova algoritmu, kde do **prioritní fronty** se místo všech uzlů vkládají pouze ty, ke kterým jsme již došli.

1. Sestroj dva **seznamy**: OPEN (prioritní fronta - uzly určené k expanzi) a CLOSED (již expandované uzly - existuje verze i bez tohoto seznamu). Do open vlož počáteční uzel včetně jeho ohodnocení.
2. Je-li seznam OPEN **prázdný**, pak úloha **nemá řešení**. Jinak pokračuj.
3. Vyber ze seznamu OPEN uzel s **nejnižším ohodnocením**.
4. Je-li vybraný uzel **cílovým**, ukonči prohledávání jako **úspěšné** a vrať cestu. Jinak pokračuj.
5. Vybraný uzel expanduj a jeho následníky, kteří nejsou v CLOSED, umísti do OPEN (**včetně jejich ohodnocení**). Expandovaný uzel vlož do CLOSED. Z uzlů, které se v OPEN vyskytují **vícekrát**, ponech jenom ten s **nejnižším ohodnocením** (jinak řečeno, aktualizuj ohodnocení, pokud je lepší). Vrať se na bod 2.

Časová i prostorová složitost je dána **cenou optimálního řešení C^*** a **minimálním přírůstkem ceny ΔC_{min}** mezi dvěma uzly:

- $O(b^{(C^*/\Delta C_{min})})$

Prohledávání se zpětným navracením - Backtracking Search

Metoda je vhodná i pro **CSP** (pokud aplikace operátoru vede na stav porušující omezující podmínky, pak je tento operátor považován za **neaplikovatelný** - bod 3 algoritmu), **je úplná**, ale **není optimální**. Backtracking je podobný DFS, ale místo expanze uzlu generuje pouze jediného následníka - nejprve jednoho a při návratech pak další.

- 1. Sestroj zásobník OPEN a umísti do něj počáteční uzel.
- 2. Je-li OPEN prázdný, pak úloha nemá řešení - ukonči prohledávání jako neúspěšné. Jinak pokračuj.
- 3. Jde-li na uzel na vrchu zásobníku aplikovat první/další operátor, tak tento operátor aplikuj a pokračuj bodem 4. Jinak odstraň testovaný uzel z vrcholu a vrať se na 2.
- 4. Je-li vygenerovaný uzel uzlem cílovým, ukonči prohledávání jako úspěšné a vrať cestu. Jinak ulož uzel na vrchol zásobníku (modifikace: uzel na zásobník ulož, pokud se tam ještě nenachází) a vrať se na 2.

Časová a prostorová složitost je stejná jako pro DFS:

- **bez modifikace**: časově i prostorově má nekonečnou časovou složitost,
- **s modifikací**: časová složitost je $O(b^m)$ a prostorová složitost je $O(m)$, kde **b** je faktor větvení a **m** je počet **různých stavů**.

Prohledávání s dopřednou kontrolou - Forward Checking Search

Vhodné i pro CSP. Metoda je **úplná** i **optimální**. forward checking, CSP heuristics.

Funguje na principu, že každé proměnné (uzlu) přiřadí množinu všech přípustných hodnot pro danou proměnnou (u barvení mapy to budou **všechny dostupné barvy**). U první proměnné vybereme některou z přípustných hodnot (např. modrou barvu) a aktualizujeme přípustné hodnoty u ostatních proměnných (proměnné, které na mapě sousedí s právě obarvenou proměnnou, nemohou být modré). Vybereme další proměnnou a jí přiřadíme hodnotu (např. zelenou) a takhle postup opakujeme dokud nejsou pro všechny proměnné vybrány hodnoty. Může se ale stát, že u některé proměnné dojde k odstranění všech možných hodnot, které ji lze přiřadit (už nelze obarvit, aby nebylo porušeno pravidlo). Pak musíme u předešlých proměnných změnit jejich přiřazení (postupně se vypočítat a měnit). Pokud již nejde přiřazení nijak změnit, úloha nemá řešení.

Heuristiky pro výběr proměnných:

- Proměnná s **nejmenším počtem přípustných hodnot**.
- Proměnná, která má **největší vliv na omezení zbývajících** volných proměnných.

Heuristiky pro přiřazení hodnoty proměnné:

- Vyber hodnotu, která **vylučuje nejméně hodnot**, které mají společná omezení s vybranou proměnnou.

Složitost **n** - počet proměnných, **m** - počet přípustných hodnot:

- časová: **$O(m^n)$** ,
- prostorová: **$O(n)$** .

Prohledávání s minimalizací konfliktů - Min-Conflict Search

Vhodné i pro CSP. **Neexistuje důkaz** o její **úplnosti** ani **optimálnosti** a tedy ani odhad časové složitosti.

1. Přiřaď každé proměnné x_i ($i = 1, \dots, n$) **libovolnou** hodnotu z množiny **přípustných** hodnot. Nastav pomocné proměnné **i** (počítadlo proměnných) a **j** (počítadlo **správně** přiřazených proměnných) na hodnoty **1**.
2. Spočítej počet **možných konfliktů** pro každou hodnotu proměnné **x_i** .
3. Pokud existuje pro jinou možnou hodnotu **x_i** počet konfliktů **menší nebo stejný** než pro její aktuální hodnotu, změň ji na tuto hodnotu a nastav hodnotu **j na 1**, jinak **inkrementuj j**.
4. Pokud **$j == n$** (všechny proměnné jsou přiřazeny nejlépe), bylo nalezeno optimální řešení. Jinak inkrementuj **i** a **pokračuj**.
5. Je-li hodnota **i > n**, pak **i = 1** a pokračuj bodem 2.

V paměti si udržuje pouze nynější stav a par proměnných, tedy prostorová náročnost je: **$O(1)$**

Informované metody

Používají heuristické funkce pro **odhad ceny** z aktuálního stavu do cíle a přičítají jej k aktuální ceně cesty. **f(sk)** je ohodnocení k-tého stavu, které je dáno součtem:

- **g(sk)**: cena cesty od **počátečního stavu** ke **k-tému stavu**.
- **h(sk)**: je odhadovaná (pomocí **heuristické** funkce) cena cesty od **k-tého stavu** ke stavu **cílovému**.

Prohledávání od nejlepšího - Best First Search (BestFS)

- Sestroj **seznam OPEN** (popř. CLOSED) a vlož do něj počáteční uzel včetně jeho ohodnocení.
- Je-li seznam OPEN prázdný, pak úloha nemá řešení - skonči s neúspěchem. Jinak pokračuj.
- Vyber z OPEN uzel s nejlepším ohodnocením.
- Je-li vybraný uzel uzlem cílovým - skonči s úspěchem a vrať cestu. Jinak pokračuj.
- Expanduj vybraný uzel. Všechny jeho následovníky co nejsou předky (využití CLOSED), umístí do OPEN včetně jejich ohodnocení. Z těch co jsou v OPEN **vícekrát** ponech pouze uzel s **nejlepším ohodnocením**. Vrať se na 2.

Neinformovaná metoda UCS je extrémním případem BestFS, u které je hodnota **h(sk)** **vždy 0**.

Chamtivé prohledávání - Greedy Search (GS)

Jedná se také o extrémní případ BestFS, kdy $g(sk) = 0$. **Úplná** (pokud se používá seznam CLOSED), ale **není optimální**.

Časová a prostorová složitost je $O(b^d)$, kde b je faktor větvení a d je hloubka.

A* prohledávání/Search

Spadá pod BestFS. Je **úplná a optimální**. Heuristická funkce je **spodním odhadem skutečné** ceny (**nejlepší/nejlevnější** odhad ceny) cesty od ohodnoceného uzlu k cíli. Tato funkce nikdy **nesmí** odhadovanou cenu cesty **přecenit**, jinak algoritmus nebude fungovat správně, $h = 0$ je přípustná vždy (ale pak je to metoda UCS).

Časová i prostorová náročnost závisí na heuristice, pohybuje se od:

- pro h blízké nule: $O(b^d)$,
- pro h rovné skutečné ceně: $O(d)$,

kde b je faktor větvení a d je hloubka cíle.

Metody lokálního prohledávání

většinou nejsou úplné ani optimální, ale mohou být například rychlé.

Hill Climbing

Není úplná ani optimální. Umí jít pouze jedním směrem a nedokáže se vracet. Pokud by měla hledat např. nejvyšší horu z údolí, je velmi pravděpodobné, že se zastaví na prvním kopci, protože neumožňuje klesání.

1. Vytvoř uzel **Current** a ulož do něj počáteční stav **s0** spolu s jeho ohodnocením.
2. Expanduj **Current**, ohodnoť jeho bezprostřední následníky a vyber z nich **nejlépe ohodnoceného (Next)**.
3. Je-li ohodnocení **Current** lepší než ohodnocení **Next** (tedy lepší než všechna ohodnocení všech bezprostředních následníků expandovaného **Current**), ukonči řešení a vrať **Current**. Jinak pokračuj.
4. Nahradi **Current** uzlem **Next**. Vrať se na 2.

Prostorová složitost je $O(1)$ - používáme pouze 2 proměnné Current a Next. Časová složitost je $O(d)$ - jdeme pouze jednou cestou.

Simulated Annealing

Není úplná ani optimální. Pracuje jak s ohodnocením, tak s **náhodností**. Narozdíl od Hill Climbing **dokáže opustit lokální extrém**. Je inspirována tuhnutím kovů. S postupně **klesající pravděpodobností** (tuhnutím kovu s klesající teplotou) umožňuje vybrat **hůře** ohodnocený uzel - na začátku algoritmu je poměrně pravděpodobné, že opustíme lokální extrém.

1. Vytvoř tabulku pro klesání "teploty" v závislosti na kroku výpočtu.
2. Vytvoř pracovní uzel Current a ulož do něj počáteční stav a jeho ohodnocení. Nastav krok výpočtu na 0 ($k=0$).
3. Z tabulky zjisti aktuální teplotu T . Je-li $T=0$, ukonči řešení a vrať jako výsledek Current. Jinak pokračuj.
4. Expanduj Current a z jeho následovníků vyber náhodně jednoho z nich (Next).

5. Vypočítej rozdíl ohodnocení uzlů Current a Next :

$$\Delta E = \text{value}(\text{Current}) - \text{value}(\text{Next}).$$

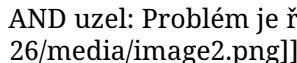
6. Pokud $\Delta E > 0$, pak nahraď uzel Current uzlem Next. Jinak zaměň uzel Next za jiný s pravděpodobností $p = e^{\Delta E/T}$.

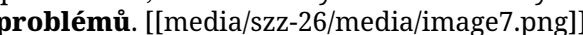
7. Inkrementuj k a vrať se na 3.

Časová složitost závisí na rychlosti klesání teploty (rychlosti klesání pravděpodobnosti, že bude vybrán hůře ohodnocený uzel). Prostorová složitost je **$O(1)$** - konstantní.

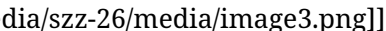
Metody řešení úloh rozkladem na podproblémy (AND/OR grafy)

U těchto úloh uzly znamenají **problémy/podproblémy**, NIKOLIV stavy. Problém lze rozložit dvěma způsoby:

- **OR uzel:** Problém je řešitelný, pokud je **alespoň jeden** z podproblémů **řešitelný**.
- **AND uzel:** Problém je řešitelný, pokud jsou **řešitelné všechny** jeho podproblémy. 

Obecný **AND/OR** graf (graf s uzly AND a OR) lze převést na graf, ve kterém jsou v každé **vrstvě** buď pouze **AND**, nebo **OR** uzly. Každou musí být možné převést ze stavového prostoru na **rozklad podproblémů**. 

Slepé AND/OR grafy pro BFS, DFS



BFS a DFS prohledávání s tím, že řešitelnost uzlu je dána jeho typem (AND, OR, případně kombinací AND a OR) a řešení spočívá v dokázání, že je **řešitelný** i **počáteční uzel** (kořen). Toho se docílí **propagováním informace o řešitelnosti/neřešitelnosti** z nižších vrstev do vyšších a na tom je algoritmus založený.

1. Sestroj OPEN (**fronta** pro BFS, **zásobník** pro DFS) a prázdný **graf/strom G** a do obou ulož počáteční uzel (problém), který **nesmí** být elementárně řešitelný nebo neřešitelný.
2. Výjmi z OPEN uzel a označ jej X.
3. Expanduj X a všechny jeho následovníky připoj ke grafu G.
 1. Pro všechny **řešitelné** následníky X přenes **informaci o jejich řešitelnosti jejich předchůdcům**. Je-li **řešitelný počáteční** problém, ukonči řešení jako **úspěšné**.
 2. Pro všechny **neřešitelné** následníky X přenes **informaci o jejich neřešitelnosti jejich předchůdcům**. **Není-li řešitelný počáteční** problém, ukonči řešení jako **neúspěšné**.
 3. Všechny ostatní následníky X (tedy ty, které nejsou elementárně řešitelné – stále jsou nevyřešené) ulož do OPEN.
4. Odstraňte z OPEN všechny uzly, které mají **vyřešené předchůdce** (nedává smysl je už řešit).
5. Je-li OPEN **prázdný** - skonči s **neúspěchem**. Jinak se vrať na 2.

Slepý AO pro Backtracking

Podobný jako DFS, test uzlu na **řešitelnost/neřešitelnost** probíhá **před** generováním jeho následníka - vždy se generuje pouze **jeden následník**, který se vyhodnotí a až poté se **případně generuje další**.

1. Sestroj **graf G** a **zásobník** OPEN a do obou ulož počáteční uzel.

2. Je-li uzel na vršku OPEN řešitelný, pak:
 1. Je-li tento uzel počáteční - skončí **úspěšně**.
 2. Jinak přenes informaci o řešitelnosti na předchůdce a uzel z OPEN odstraň.
3. Je-li uzel na vršku neřešitelný, pak:
 1. Je-li tento uzel uzlem počátečním, ukonči řešení jako **neúspěšné**.
 2. Jinak přenes informaci o neřešitelnosti na předchůdce, a uzel odstraň z OPEN.
4. Není-li uzel na vršku OPEN řešitelný/neřešitelný, pak **generuj** jeho následníka, ulož ho do OPEN a připoj k G.
5. Vrať se na bod 2.

Informovaný AO* algoritmus

Algoritmus řeší, jestli je uzel řešitelný a také jaká je cena vyřešení uzlu. Může skončit neúspěšně, i když existuje řešení, pokud je převýšena jeho **maximální cena**. Výpočet ceny pro uzly je (může být doplněno o heuristickou funkci):

- Ohodnocení **AND** je rovno **součtu** ohodnocení všech jeho následníků.
- Ohodnocení **OR** je rovno **nejmenšímu** z ohodnocení jeho následníků.

Algoritmus pracuje následovně:

1. Sestroj strukturu G (AO strom) a umísti do něj počáteční uzel (INIT) s ohodnocením. Stanov hodnotu FUTILITY (maximální povolenou cenu).
2. Je-li INIT označen jako řešitelný (SOLVED), ukonči řešení jako úspěšné. Je-li $INIT \geq FUTILITY$, skonči s neúspěchem. Jinak pokračuj.
3. Procházej nejnadějnější podstrom až narazíš na neexpandovaný uzel (NODE).
4. Expanduj NODE, pokud nemá následníka, přiřaď mu hodnotu FUTILITY a přejdi na 7. Jinak pokračuj.
5. Představují-li někteří bezprostřední následníci elementární úlohy, označ je SOLVED (0 ohodnocení) u zbývajících jej vypočti.
6. Připoj následníky NODE ke stromu G a přenes informaci o jejich ohodnocení směrem k INIT.
7. V uzlech OR označ nejnadějnější podstomy.
8. Vrať se na bod 2.

graficky viz: <https://www.goeduhub.com/7063/implement-ao-search-algorithm>

Metody hraní her

Cílem je **určit tah hráče na tahu**, který **povede k vítězství**, nebo pro který je **pravděpodobnost** jeho vítězství největší. Nejprve budeme uvažovat pouze hry, které hrají dva pravidelně se střídající hráči, které označíme symboly **A** a **B**. Každý z nich chce vyhrát a oba mají **úplný** přehled o aktuálním stavu hry. Pro tyto hry obecně platí:

- Hráč na tahu (označuje se vždy jako hráč **A**) zvítězí, vede-li k jeho vítězství **alespoň jeden** tah □ problém **OR**.
- Hráč na tahu zvítězí, vedou-li po jeho tahu k jeho vítězství **všechny možné tahy protihráče** (hráče **B**) □ problém **AND**.

Rozdělení her dvou protihráčů

- **Jednoduché hry** - Lze prozkoumat všechny možné tahy (hra NIM). K řešení lze použít **AND/OR** algoritmy.
- **Složité hry** - Zkoumá se pouze několik následujících tahů (šachy).
- **Hry s neurčitostí** - (hod kostkou např.) Zkoumá se pouze několik následujících tahů s respektováním **neurčitosti** (hod kostkou - člověče, nezlob se). Pracuje se s očekávanými hodnotami.

MiniMax (Složité hry)

Každý stav je ohodnocen hodnotící funkcí (kladná příznivé pro hráče na tahu - **A**, vítězství/prohra má hodnotu $+\infty$). Hráč **A** chce **maximální** hodnoty - v **OR** uzlu se bere **max** jeho potomků, **B** chce **minimální** - v **AND** uzlu se bere min jeho potomků. Řešením problému v daném stavu je určení **nejvýhodnějšího tahu hráče A** (pro každý tah se musí řešit znovu a znovu). Metoda je **rekurzivní** a zahajuje se, když je **na tahu hráč A**. Vstupem jsou

- stavy hry X,
- maximální hloubka prohledávání.

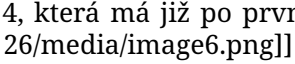
Výstupem je

- **aktuální stav**,
- **tah**, který k tomuto stavu vede.

Algoritmus může vypadat následovně

1. Je-li uzel **X** listem, vrací ohodnocení tohoto uzlu.
2. Je-li na tahu hráč **A**, postupně pro všechny jeho možné tahy volá **MiniMax** pro hráče B a vrací **maximální** z navracených hodnot a tah, který k tomuto vede.
3. Je-li na tahu hráč **B**, tak postupně pro všechny jeho možné tahy (bezprostřední následníky uzlu X, tj. stavy před tahem hráče A) volá proceduru **MiniMax** pro hráče A a vrací **minimální** z navracených hodnot.

Popis příkladu na obrázku:

- žlutá čísla určují **pořadí** při vyhodnocování,
- nepodbarvená čísla určují **ohodnocení uzlů** a změny jejich ohodnocení,
- **červené** uzly jsou vyhodnocovány **zbytečně** (už je jasné, že si hráč tuto větev stejně nezvolí. V kroku 4 je jasné, že hráč **B** si **radši** vybere větev kroku 3 (ohodnocení 8 < 9), než větev 4, která má již po prvním potomkovi ohodnocení 9 a už může mít pouze vyšší. 

AlfaBeta (Složité hry)

Step by Step: Alpha Beta Pruning

Zbytečnému vyšetřování uzlů lze zabránit pomocí alfa-beta řezů. **Alfa** řezy zabraňují zbytečnému vyšetřování tahů hráče **A**, **beta** řezy pak zbytečnému vyšetřování tahů hráče **B**. Procedura AlfaBeta vychází z procedury MiniMax, je rekurzivní, vstupními parametry jsou stejné a navíc přidává parametry α a β , které jsou na začátku nastaveny na $\alpha = -\infty$ a $\beta = +\infty$ (tedy opačné hodnoty než hráči A a B požadují).

1. Je-li uzel X listem, procedura vrací ohodnocení tohoto uzlu.
2. Je-li **na tahu A**:

1. Dokud platí $\alpha < \beta$, tak postupně pro všechny tahy volá **AlfaBeta** s aktuálními hodnotami α a β (po každém volání se α mění). Po **každém** vyšetření tahu **nastaví α na maximum** (postupně se zvětšuje z $-\infty$) z aktuální a vrácené hodnoty.
2. Je-li $\alpha \geq \beta$ (před každým dalším následníkem se **musí toto kontrolovat**), nebo nemá-li **X žádného** dalšího bezprostředního **následníka**, vrací aktuální hodnotu α a tah, který vede k nejlepšímu stavu (v případě více stejných ten první nalezený).

3. Je-li **na tahu B**:

1. Pokud platí $\alpha < \beta$, tak postupně pro tahy volá **AlfaBeta** s aktuálními α a β (po každém volání se β mění). Po každém vyšetření nastaví β na **minimum** (postupně se zmenšuje z ∞) z aktuální a vrácené hodnoty.
2. Je-li $\alpha \geq \beta$ nebo nemá-li **X žádné následníky**, vrací **aktuální β** .

Popis příkladu na obrázku:

- žlutá čísla určují **pořadí** při vyhodnocování,
- nepodbarvená čísla určují **ohodnocení α a β** a změny jejich ohodnocení,

ExpectiMiniMax (Hry s neurčitostí)

[[media/szz-26/media/image8.png]]

Games: Q2. Expectiminimax AI U3 D5 Probabilistic Cut ExpectiMax AlphaBeta

Podobné MinMax, počítá se ale z pravděpodobnosti, že k nějakému stavu dojde. Metoda je rekurzivní a pracuje s **expectimin** pro hráče **B** (očekávané **minimum**) a **expectimax** pro hráče **A** (očekávané **maximum**). **Expectimin** a **expectimax** jsou spočteny jako **součet ohodnocení po všech možných výsledcích hodu kostky** (případně jiné pravděpodobnostní veličiny). Např. pokud má uzel 2 následovníky, první má ohodnocení 10, druhý má ohodnocení 5. Pravděpodobnost vybrání 1. následníka je ale pouze 0.25, druhého 0.75. celkové ohodnocení uzlu tak bude $0.25 \cdot 10 + 0.75 \cdot 5 = 6,25$. [[media/szz-26/media/image9.png]]

1. Je-li uzel **X** listem (konečným stavem hry, nebo uzlem v **maximální hloubce**) procedura vrací **ohodnocení tohoto uzlu**.
2. Je-li na tahu hráč **A**, postupně pro všechny jeho možné tahy (bezprostřední následníky uzlu **X**) volá proceduru **Expectiminimax** pro hráče **B**, vrací **maximální** hodnotu z navrácených hodnot **expectimin** a tah, který k nejlépe ohodnocenému bezprostřednímu následníkovi **vede** (tento **tah** má opět význam pouze u kořenového uzlu, kdy představuje nejvýhodnější reálný tah hráče **A**).
3. Je-li na tahu hráč **B**, postupně pro všechny jeho možné tahy (bezprostřední následníky uzlu **X**) volá proceduru **Expectiminimax** pro hráče **A** a vrací **minimální** hodnotu z navrácených hodnot **expectimax**.

Metody hraní her n hráčů

Hráči se pravidelně střídají, každý hráč chce vyhrát a všichni mají úplný přehled o aktuálním stavu hru.

Jednotlivé stavy hry jsou ohodnocovány **jedinou hodnotící funkcí** aplikovanou na **každého hráče zvlášť**, přičemž hodnota této funkce musí být **tím vyšší**, čím je daná pozice pro hodnoceného hráče **výhodnější**. U každého stavu je pak jeho ohodnocení dáno **seznamem hodnot hodnotící funkce** pro jednotlivé hráče. Například pro hru se třemi hráči ohodnocení stavu **(8 3 5)** znamená, že daný stav má ohodnocení **8 pro prvního** hráče, **3 pro druhého** hráče a **5 pro třetího** hráče. Každý hráč si

vybírání pro něj nejvýhodnější ohodnocení. Na obrázku jsou zeleně vyznačena ohodnocení pro hráče 1 a modře ohodnocení pro hráče 2.

Procedura Max^n (hry více hráčů)

[[media/szz-26/media/image4.png]]

Jedná se o zobecnění procedury **MiniMax** pro n hráčů. Proceduru volá vždy hráč na tahu

- vstup: aktuální stav hry (uzel X)
- výstup: ohodnocení tohoto stavu

Princip algoritmu:

1. Je-li uzel X listem (konečným stavem hry nebo uzlem v maximální hloubce), tak procedura vrací ohodnocení tohoto uzlu ($v_1, v_2, \dots, v_n$).
2. Jinak tato procedura pro aktuálního hráče i volá rekurzivně sama sebe na všechny své bezprostřední následníky (tj. stavy před tahem následujícího hráče j ($j = i + 1$ a **pokud** $j > n$, **pak** $j = 1$) a vrací ohodnocení, které je pro daného hráče, tj hráče i , nejvýhodnější.

Soft- Max^n

Problémem procedury **Max** ^{n} je nejednoznačnost hodnocení uzlu (stavu hry) v případech, kdy stejné **maximální ohodnocení má několik** jeho bezprostředních následníků. Tento problém procedura Soft- Max^n řeší: vrací množiny se stejnými ohodnoceními pro příslušného hráče, a tím umožňuje hráči na tahu informovanější rozhodování. [[media/szz-26/media/image1.png]]

Pokud **hráči 1** stačí k vítězství **4** body, pak si tento hráč zřejmě vybere tah rovně dolů k uzlu **(4 6 5)**. V opačném případě může buď **riskovat**, tj. zvolit tah vlevo (hráč **2** si však může vybrat tah (1 6 4), který je pro hráče 1 velmi nevýhodný - hodnota 1), nebo sázet raději na jistotu, tj. zvolit tah vpravo. [[media/szz-26/media/image10.png]]

Odkazy:

- Tabulka metod ve větším: <https://cdn.discordapp.com/attachments/539908031157370900/577122492934520833>
- Metody prohledávání

Zdroje

- SZZ okruh 26 — studijní materiály FIT BUT (szz-26.docx). Obrázky: media/szz-26/.

Navigace: [[index| • Přehled okruhů]] · □ [[okruhy/25-teorie-grafu|25. Teorie grafů]] · další: [[okruhy/27-strojove-uceni|27. Strojové učení]] □